Computational Problems and Algorithms

CSALGCM

Prepared by: Austin Fernandez

Agenda

1. Definition of a Computational Problem
2. Definition of an Algorithm
3. Types of Problems
4. Algorithm Correctness
5. Algorithm Complexity
6. Warm-Up Problem Set

What is a Computational Problem?

Definition of Computational Problem

- A computational problem is defined by its input and output.
- Both input and output have specifications.
- The output’s specifications are relative to the input.

Examples

SORT-SEQUENCE

Input: A sequence of n integers $a_1, a_2, \ldots, a_n$

Output: A permutation of the input integers $b_1, b_2, \ldots, b_n$

such that for all $i > 1$, $b_{i+1} >= b_i$.

Examples

COUNT-FACTORS

Input: A positive integer n

Output: An integer m denoting that there are m distinct positive integers k such that k divides n

How do you solve a computational problem?

...using an algorithm!

Definition of an algorithm

It is a sequence of finite and unambiguous instructions that operates on the input of a computational problem and uses it to produce the output that meets the problem’s specifications.

Solution to SORT-SEQUENCE

```
procedure BUBBLE-SORT(arr : int[], n : int)
// takes an array of integers arr and its size, n
// returns a permutation of arr which is non-decreasing

    for i from n downto 2
        for j from 1 to i - 1
            if arr[j] > arr[j + 1]
                swap arr[j] and arr[j + 1]
    return arr
```

Solution to COUNT-FACTORS

```
procedure COUNT-FACTORS(n : int)
// takes an integer n
// returns an integer denoting the number of factors of n

    i <- 1
    ctr <- 0
    while i <= n
        if n mod i = 0
            ctr <- ctr + 1
        i <- i + 1
    return ctr
```

Types of Problems

1. Decision
2. Search
3. Counting
4. Optimization
5. Function

Decision Problem

- Output is “YES” or “NO”
- “Decide” if a certain property applies to the input.
- Examples
  - Is an integer **N** a prime number?
  - Is a string **S** alphanumeric?

Search Problem

- Look for a solution to a problem that satisfies specific constraints.
- Examples
  - Sudoku
  - Pathfinding
  - 8-Puzzle
  - Rubik’s cube

Counting Problem

- Count how many solutions to a problem that satisfies specific constraints.
- Examples
  - Counting paths in a DAG
  - Coin Change

Optimization Problem

- Look for a solution to a problem that satisfies specific constraints and is the best solution based on some criterion or measure.
- Examples
  - Single-Source Shortest Path
  - Minimum Spanning Tree
  - Minimum Element in an Array
  - Huffman Encoding

Function Problem

- Compute a formally defined function.
- Examples
  - Computing mean or standard deviation
  - Euler Phi Function

How do we analyze an algorithm?

Algorithm Analysis

1. Correctness
2. Efficiency
    a. Time
    b. Space
3. Elegance

Algorithm Correctness

- Does the algorithm produce output that meets the problem’s output specifications.
- Might require mathematical proof.
- Some problems are too complex to prove correctness.
- These problems can be tested using software evaluation techniques.

Algorithm Efficiency

- Efficiency describes the resources an algorithm requires to finish execution.
- Time Complexity
  - Time required to finish execution
- Space Complexity
  - Memory required to finish execution.

Algorithm Elegance

- Elegance is the simplicity of an algorithm’s logic.
- Occam’s Razor

Thank You!

References

Cormen, T. H., Leiserson, C. E., & Rivest, R. L., Stein, C. (2009). Introduction to Algorithms. 3rd Edition. MA: The MIT Press.

Dasgupta, S., Papadimitriou, C. H., & Vazirani, U. V. (2008). Algorithms. McGraw-Hill Higher Education.